

Funciones vs Métodos

Por: Ricardo Cuellar



Datos importantes

En Go, un método es una función con receiver.

Función = Libre

Método = Asociado a un tipo



¿Cómo se ve un receiver?



methods.go

```
func F(x int) int { return x*2 }
```

```
type Counter struct{ n int }
```

```
func (c *Counter) Inc() { c.n++ }
```



¿Qué gana Go con los métodos?

1. Organización

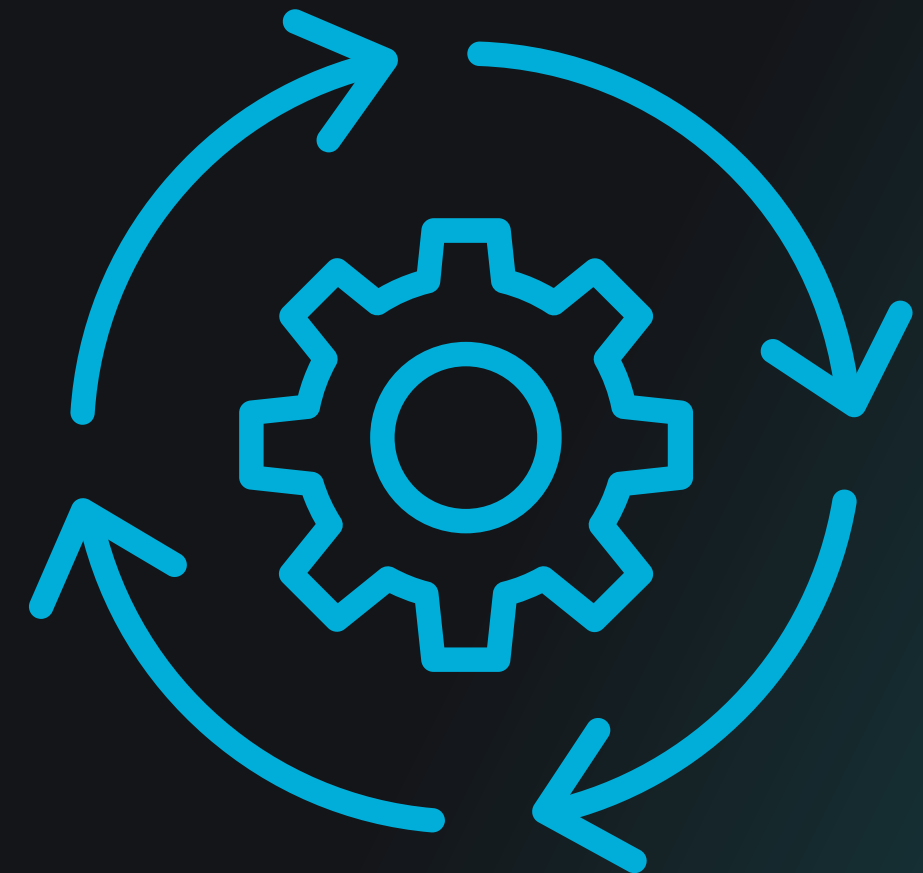
a. Comportamiento cerca de los datos

2. Interfaces

a. Si un tipo tiene los métodos, cumple la interfaz

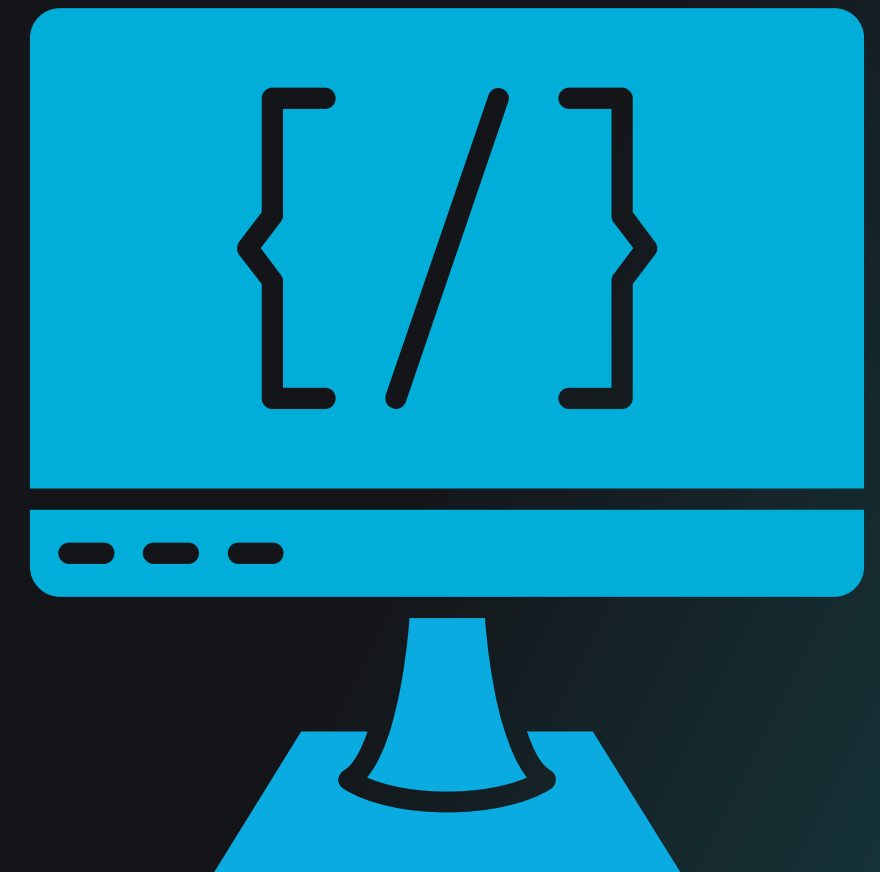
3. Encapsulación

a. API del tipo



¿Cuándo uso la función?

- Utilidades / Algoritmos
- Reglas inyectables
- Cosas muy generales
- Calculos
- Funciones para inyectar



¿Cuándo uso los métodos?

- Operaciones propias del tipo
- Mutación
- Estandarizar comportamientos de un tipo



methods.go

```
func (o Order) HasSKU(sku string) bool
```

```
func (o Order) Subtotal() Money
```

```
func (o *Order) AddItem(it Item)
```



Algunos consejos

- Si suena como “Order hace X” elige un método.
- Si suena como “Calcula X cosa usando Order” elige función
- Si necesitas una interfaz elige método
- Si necesitas inyección elige función



Recuerda

Funciones son comportamiento suelto. Métodos son comportamiento pegado a un tipo. En Go, las interfaces se cumplen con métodos, por eso los métodos son la puerta a polimorfismo."



Gracias



www.devtalles.com



[/ricardocuellars](https://www.linkedin.com/company/ricardocuellars)



[@ricardocuellars](https://twitter.com/ricardocuellars)

